1. Use ROW_NUMBER to assign a unique ID to each customer from the 'customer' table ordered by 'customer_first_name', and return only those whose name starts with 'A'.

```
SELECT *
FROM (
  SELECT *,
       ROW_NUMBER() OVER (ORDER BY customer_first_name) AS row_id
  FROM customer
) AS sub
WHERE customer_first_name LIKE 'A%';
```

---

2. For each customer from 'customer_purchases' and 'customer', find their first purchase date using ROW_NUMBER, and display the corresponding 'product_id' and 'quantity'.

```
SELECT *
FROM (
  SELECT cp.*, c.customer_first_name, c.customer_last_name,
       ROW_NUMBER() OVER (PARTITION BY cp.customer_id ORDER BY cp.market_date) AS rn
  FROM customer_purchases cp
  JOIN customer c ON cp.customer_id = c.customer_id
) AS sub
WHERE rn = 1;
```

---

3. Use DENSE_RANK to rank vendors from the 'vendor' and 'vendor_booth_assignments' tables by number of booth assignments.

```
SELECT vendor_id,
     COUNT(*) AS booth_count,
     DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS vendor_rank
```

```
FROM vendor_booth_assignments
GROUP BY vendor_id;
```

---

4. Retrieve the top 3 most frequently purchased products per customer using RANK with joins between 'customer_purchases', 'product', and 'customer'.

```
SELECT *
FROM (
  SELECT cp.customer_id,
       cp.product_id,
       SUM(cp.quantity) AS total_quantity,
       RANK() OVER (PARTITION BY cp.customer_id ORDER BY SUM(cp.quantity) DESC) AS rnk
  FROM customer_purchases cp
  GROUP BY cp.customer_id, cp.product_id
) AS sub
WHERE rnk <= 3;
```

---

5. Use ROW_NUMBER to identify duplicate vendor names from the 'vendor' table after applying TRIM, and keep only the first occurrence.

```
SELECT *
FROM (
  SELECT *,
       ROW_NUMBER() OVER (PARTITION BY TRIM(vendor_name) ORDER BY vendor_id) AS rn
  FROM vendor
) AS sub
WHERE rn = 1;
```

---

6. Extract the first 3 letters of each customer's name from 'customer' using LEFT and assign a row number partitioned by ZIP code.

```
SELECT customer_id,
    LEFT(customer_first_name, 3) AS name_prefix,
    customer_zip,
    ROW_NUMBER() OVER (PARTITION BY customer_zip ORDER BY customer_first_name) AS zip_row
FROM customer;
```

---

7. Rank vendors by revenue using DENSE_RANK from 'vendor', 'customer_purchases', and 'product'.

```
SELECT cp.vendor_id,
    SUM(cp.quantity * cp.cost_to_customer_per_qty) AS total_revenue,
    DENSE_RANK() OVER (ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC) AS vendor_rank
FROM customer_purchases cp
GROUP BY cp.vendor_id;
```

---

8. Display each customer's first and last purchase using ROW_NUMBER on 'customer_purchases' joined with 'customer'.

```
-- First Purchase
SELECT *
FROM (
  SELECT cp.*, c.customer_first_name, c.customer_last_name,
      ROW_NUMBER() OVER (PARTITION BY cp.customer_id ORDER BY cp.market_date) AS rn
  FROM customer_purchases cp
  JOIN customer c ON cp.customer_id = c.customer_id
) AS sub
WHERE rn = 1;
```

-- Last Purchase (Just change ORDER BY to DESC)

---

9. Create a product short code using CONCAT and rank based on quantity sold, using data from 'product' and 'customer_purchases'.

```
SELECT cp.product_id,
    CONCAT(LEFT(p.product_name, 3), p.product_id) AS short_code,
    SUM(cp.quantity) AS total_quantity,
    RANK() OVER (ORDER BY SUM(cp.quantity) DESC) AS quantity_rank
FROM customer_purchases cp
JOIN product p ON cp.product_id = p.product_id
GROUP BY cp.product_id, p.product_name, p.product_id;
```

---

10. Rank booths from 'booth' by number of vendor assignments in 'vendor_booth_assignments'.

```
SELECT vba.booth_number,
    b.booth_type,
    COUNT(*) AS assign_count,
    RANK() OVER (ORDER BY COUNT(*) DESC) AS booth_rank
FROM vendor_booth_assignments vba
JOIN booth b ON vba.booth_number = b.booth_number
GROUP BY vba.booth_number, b.booth_type;
```

11. For each vendor, use ROW_NUMBER to retrieve their most recent booth assignment by market_date and display the booth description.

```
SELECT *
FROM (
```

```
 SELECT vba.vendor_id,
        vba.booth_number,
        b.booth_type,
        vba.market_date,
        ROW_NUMBER() OVER (PARTITION BY vba.vendor_id ORDER BY vba.market_date DESC) AS rn
 FROM vendor_booth_assignments vba
 JOIN booth b ON vba.booth_number = b.booth_number
) AS sub
WHERE rn = 1;
```

---

12. Use RANK to assign a ranking to each product within its category based on the total quantity purchased.

```
SELECT *
FROM (
 SELECT p.product_id,
        p.product_name,
        p.product_category_id,
        SUM(cp.quantity) AS total_quantity,
        RANK() OVER (PARTITION BY p.product_category_id ORDER BY SUM(cp.quantity) DESC) AS
category_rank
 FROM customer_purchases cp
 JOIN product p ON cp.product_id = p.product_id
 GROUP BY p.product_id, p.product_name, p.product_category_id
) AS sub;
```

---

13. Combine CASE and RANK to categorize customers as 'Top Buyer', 'Average Buyer', or 'Low Buyer' based on purchase quantity rankings.

```
SELECT *,
```

```
    CASE
      WHEN rnk = 1 THEN 'Top Buyer'
      WHEN rnk = 2 THEN 'Average Buyer'
      ELSE 'Low Buyer'
    END AS buyer_category
FROM (
  SELECT cp.customer_id,
      SUM(cp.quantity) AS total_quantity,
      RANK() OVER (ORDER BY SUM(cp.quantity) DESC) AS rnk
  FROM customer_purchases cp
  GROUP BY cp.customer_id
) AS sub;
```

---

14. For each product category, rank customers based on total quantity purchased using DENSE_RANK.

```
SELECT *
FROM (
  SELECT p.product_category_id,
      cp.customer_id,
      SUM(cp.quantity) AS total_quantity,
      DENSE_RANK() OVER (PARTITION BY p.product_category_id ORDER BY SUM(cp.quantity) DESC)
AS category_rank
  FROM customer_purchases cp
  JOIN product p ON cp.product_id = p.product_id
  GROUP BY p.product_category_id, cp.customer_id
) AS sub;
```

---

15. Use ROW_NUMBER to return the most recent inventory update per product for each vendor.

```
SELECT *
FROM (
  SELECT vi.*,
       ROW_NUMBER() OVER (PARTITION BY vi.vendor_id, vi.product_id ORDER BY vi.uploaded_at
DESC) AS rn
  FROM vendor_inventory vi
) AS sub
WHERE rn = 1;
```

---

16. Join vendor and booth tables, and rank vendors within each booth_type by number of booth assignments.

```
SELECT vendor_id,
      booth_type,
      COUNT(*) AS assignment_count,
      RANK() OVER (PARTITION BY booth_type ORDER BY COUNT(*) DESC) AS vendor_rank
FROM vendor_booth_assignments vba
JOIN booth b ON vba.booth_number = b.booth_number
GROUP BY vendor_id, booth_type;
```

---

17. For each vendor, rank the original prices of their products using ROW_NUMBER and display only those with a product size ending in 'g'.

```
SELECT *
FROM (
  SELECT vi.vendor_id,
       p.product_name,
       p.product_size,
       vi.original_price,
       ROW_NUMBER() OVER (PARTITION BY vi.vendor_id ORDER BY vi.original_price DESC) AS rn
```

```
  FROM vendor_inventory vi
  JOIN product p ON vi.product_id = p.product_id
  WHERE p.product_size LIKE '%g'
) AS sub;
```

---

18. Create a revenue tier per vendor using CASE and rank them using RANK.

```
SELECT *,
      CASE
         WHEN total_revenue > 1000 THEN 'High'
         WHEN total_revenue > 500 THEN 'Medium'
         ELSE 'Low'
      END AS revenue_tier
FROM (
  SELECT vendor_id,
        SUM(quantity * cost_to_customer_per_qty) AS total_revenue,
        RANK() OVER (ORDER BY SUM(quantity * cost_to_customer_per_qty) DESC) AS revenue_rank
  FROM customer_purchases
  GROUP BY vendor_id
) AS sub;
```

---

19. Use LEFT, CASE, and RANK to identify customers whose names begin with 'J' and who are in the top 2 ZIP codes by purchase count.

```
SELECT *
FROM (
  SELECT c.customer_id,
        c.customer_first_name,
        c.customer_zip,
```

```
    COUNT(cp.purchase_id) AS purchase_count,
    RANK() OVER (PARTITION BY c.customer_zip ORDER BY COUNT(cp.purchase_id) DESC) AS
zip_rank
  FROM customer c
  JOIN customer_purchases cp ON c.customer_id = cp.customer_id
  WHERE LEFT(c.customer_first_name, 1) = 'J'
  GROUP BY c.customer_id, c.customer_first_name, c.customer_zip
) AS sub
WHERE zip_rank <= 2;
```

---

20. Retrieve the top 2 selling products per booth_type using RANK, and join with booth assignments.

```
SELECT *
FROM (
  SELECT b.booth_type,
    vi.product_id,
    SUM(cp.quantity) AS total_quantity,
    RANK() OVER (PARTITION BY b.booth_type ORDER BY SUM(cp.quantity) DESC) AS product_rank
  FROM vendor_booth_assignments vba
  JOIN booth b ON vba.booth_number = b.booth_number
  JOIN vendor_inventory vi ON vba.vendor_id = vi.vendor_id
  JOIN customer_purchases cp ON cp.product_id = vi.product_id AND cp.vendor_id = vi.vendor_id
  GROUP BY b.booth_type, vi.product_id
) AS sub
WHERE product_rank <= 2;
```

21. Use DENSE_RANK to rank vendors by cumulative revenue on rainy market dates only.

```
SELECT v.vendor_id,
```

```
    v.vendor_name,
    SUM(cp.quantity * cp.cost_to_customer_per_qty) AS total_revenue,
    DENSE_RANK() OVER (ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC) AS
rain_rank
FROM customer_purchases cp
JOIN market_date_info mdi ON cp.market_date = mdi.market_date
JOIN vendor v ON cp.vendor_id = v.vendor_id
WHERE mdi.market_rain_flag = 1
GROUP BY v.vendor_id, v.vendor_name;
```

---

22. Use ROW_NUMBER to retrieve the latest sale for each product and flag the top sale per vendor using CASE.

```
SELECT *,
    CASE WHEN rn = 1 THEN 'Top Sale' ELSE '' END AS sale_flag
FROM (
  SELECT cp.*,
      ROW_NUMBER() OVER (PARTITION BY cp.product_id, cp.vendor_id ORDER BY cp.market_date
DESC) AS rn
  FROM customer_purchases cp
) AS sub;
```

---

23. Combine TRIM, LEFT, and RANK to identify vendor name variants and rank them by inventory value.

```
SELECT *,
    RANK() OVER (ORDER BY total_value DESC) AS value_rank
FROM (
  SELECT TRIM(LEFT(vendor_name, 10)) AS short_name,
      SUM(original_price) AS total_value
```

```
  FROM vendor_inventory vi
  JOIN vendor v ON vi.vendor_id = v.vendor_id
  GROUP BY TRIM(LEFT(vendor_name, 10))
) AS sub;
```

---

24. Rank all booths by the number of unique vendors assigned to them.

```
SELECT booth_number,
    COUNT(DISTINCT vendor_id) AS vendor_count,
    RANK() OVER (ORDER BY COUNT(DISTINCT vendor_id) DESC) AS booth_rank
FROM vendor_booth_assignments
GROUP BY booth_number;
```

---

25. For each customer, assign a ROW_NUMBER to their purchases and create a 'repeat_buyer_flag' using CASE.

```
SELECT *,
    CASE WHEN rn > 1 THEN 'Repeat Buyer' ELSE 'First Purchase' END AS repeat_buyer_flag
FROM (
  SELECT cp.*,
      ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY market_date) AS rn
  FROM customer_purchases cp
) AS sub;
```

---

26. Display the top 3 vendors who sold to customers with ZIP = '22801', using RANK and join with customer table.

```
SELECT *
FROM (
```

```
 SELECT cp.vendor_id,
        COUNT(*) AS total_sales,
        RANK() OVER (ORDER BY COUNT(*) DESC) AS vendor_rank
 FROM customer_purchases cp
 JOIN customer c ON cp.customer_id = c.customer_id
 WHERE c.customer_zip = '22801'
 GROUP BY cp.vendor_id
) AS sub
WHERE vendor_rank <= 3;
```

---

27. Create a derived column for product sales tier using CASE and rank products within each tier by revenue.

```
SELECT *,
      CASE
         WHEN total_revenue >= 1000 THEN 'High'
         WHEN total_revenue >= 500 THEN 'Medium'
         ELSE 'Low'
      END AS sales_tier
FROM (
 SELECT cp.product_id,
        SUM(cp.quantity * cp.cost_to_customer_per_qty) AS total_revenue,
        RANK() OVER (ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC) AS
revenue_rank
 FROM customer_purchases cp
 GROUP BY cp.product_id
) AS sub;
```

---

28. Use ROW_NUMBER to flag the earliest booth assignment per vendor.

```
SELECT *
FROM (
  SELECT vba.*,
       ROW_NUMBER() OVER (PARTITION BY vendor_id ORDER BY market_date) AS rn
  FROM vendor_booth_assignments vba
) AS sub
WHERE rn = 1;
```

---

29. Rank vendors by number of unique products sold and filter to include only those who sold 'Organic' products.

```
SELECT vi.vendor_id,
      COUNT(DISTINCT vi.product_id) AS unique_products,
      RANK() OVER (ORDER BY COUNT(DISTINCT vi.product_id) DESC) AS product_rank
FROM vendor_inventory vi
JOIN product p ON vi.product_id = p.product_id
WHERE p.product_name LIKE '%Organic%'
GROUP BY vi.vendor_id;
```

---

30. Assign a ROW_NUMBER to every customer purchase, then filter for every customer's second-highest quantity purchase.

```
SELECT *
FROM (
  SELECT cp.*,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY quantity DESC) AS rn
  FROM customer_purchases cp
) AS sub
WHERE rn = 2;
```

31. For each vendor and month, rank products sold by revenue using RANK(), and include only those with booth_type = 'Large'.

```
SELECT *
FROM (
  SELECT vi.vendor_id,
        vi.product_id,
        DATE_FORMAT(cp.market_date, '%Y-%m') AS sale_month,
        SUM(cp.quantity * cp.cost_to_customer_per_qty) AS monthly_revenue,
        RANK() OVER (PARTITION BY vi.vendor_id, DATE_FORMAT(cp.market_date, '%Y-%m')
                ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC) AS rnk
  FROM vendor_inventory vi
  JOIN customer_purchases cp ON vi.vendor_id = cp.vendor_id AND vi.product_id = cp.product_id
  JOIN vendor_booth_assignments vba ON vi.vendor_id = vba.vendor_id
  JOIN booth b ON vba.booth_number = b.booth_number
  WHERE b.booth_type = 'Large'
  GROUP BY vi.vendor_id, vi.product_id, DATE_FORMAT(cp.market_date, '%Y-%m')
) AS sub;
```

---

32. Using RANK, find the top 2 booth types per vendor that resulted in the highest revenue.

```
SELECT *
FROM (
  SELECT vba.vendor_id,
        b.booth_type,
        SUM(cp.quantity * cp.cost_to_customer_per_qty) AS revenue,
        RANK() OVER (PARTITION BY vba.vendor_id ORDER BY SUM(cp.quantity *
cp.cost_to_customer_per_qty) DESC) AS booth_rank
  FROM vendor_booth_assignments vba
```

```
  JOIN booth b ON vba.booth_number = b.booth_number
  JOIN customer_purchases cp ON vba.vendor_id = cp.vendor_id
  GROUP BY vba.vendor_id, b.booth_type
) AS sub
WHERE booth_rank <= 2;
```

---

33. Generate a leaderboard using DENSE_RANK for customer-product pairs that generated more than ₹100 in revenue.

```
SELECT *
FROM (
  SELECT cp.customer_id,
        cp.product_id,
        SUM(cp.quantity * cp.cost_to_customer_per_qty) AS revenue,
        DENSE_RANK() OVER (ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC) AS leaderboard_rank
  FROM customer_purchases cp
  GROUP BY cp.customer_id, cp.product_id
  HAVING revenue > 100
) AS sub;
```

---

34. Use LEFT, CASE, and RANK to group products by initial character and rank them within each group by quantity sold.

```
SELECT *,
      LEFT(product_name, 1) AS initial
FROM (
  SELECT p.product_name,
        SUM(cp.quantity) AS total_quantity,
```

```
    RANK() OVER (PARTITION BY LEFT(p.product_name, 1) ORDER BY SUM(cp.quantity) DESC) AS
name_rank
  FROM product p
  JOIN customer_purchases cp ON p.product_id = cp.product_id
  GROUP BY p.product_name
) AS sub;
```

---

35. Using a join with market_date_info, rank market days with snow by product sales, and return the top 1 per product.

```
SELECT *
FROM (
  SELECT cp.product_id,
      cp.market_date,
      SUM(cp.quantity) AS snow_sales,
      RANK() OVER (PARTITION BY cp.product_id ORDER BY SUM(cp.quantity) DESC) AS snow_rank
  FROM customer_purchases cp
  JOIN market_date_info mdi ON cp.market_date = mdi.market_date
  WHERE mdi.market_snow_flag = 1
  GROUP BY cp.product_id, cp.market_date
) AS sub
WHERE snow_rank = 1;
```

---

36. Identify the top-selling product per vendor per booth_type using RANK and join across product, vendor_inventory, and booth.

```
SELECT *
FROM (
  SELECT vi.vendor_id,
      b.booth_type,
```

```
    cp.product_id,
    SUM(cp.quantity) AS total_quantity,
    RANK() OVER (PARTITION BY vi.vendor_id, b.booth_type ORDER BY SUM(cp.quantity) DESC) AS prod_rank
  FROM vendor_inventory vi
  JOIN customer_purchases cp ON vi.vendor_id = cp.vendor_id AND vi.product_id = cp.product_id
  JOIN vendor_booth_assignments vba ON vi.vendor_id = vba.vendor_id
  JOIN booth b ON vba.booth_number = b.booth_number
  GROUP BY vi.vendor_id, b.booth_type, cp.product_id
) AS sub
WHERE prod_rank = 1;
```

---

37. Rank vendors by their average booth price level and only include those with more than 5 assignments.

```
SELECT *
FROM (
  SELECT vba.vendor_id,
    AVG(b.booth_price_level) AS avg_price,
    COUNT(*) AS assignment_count,
    RANK() OVER (ORDER BY AVG(b.booth_price_level) DESC) AS price_rank
  FROM vendor_booth_assignments vba
  JOIN booth b ON vba.booth_number = b.booth_number
  GROUP BY vba.vendor_id
  HAVING COUNT(*) > 5
) AS sub;
```

---

38. Use ROW_NUMBER to find each vendor's last 2 assigned booths and create a flag if the booth_type was 'Premium'.

```
SELECT *,
      CASE WHEN booth_type = 'Premium' THEN 'Premium Booth' ELSE '' END AS flag
FROM (
  SELECT vba.vendor_id,
        b.booth_number,
        b.booth_type,
        vba.market_date,
        ROW_NUMBER() OVER (PARTITION BY vba.vendor_id ORDER BY vba.market_date DESC) AS rn
  FROM vendor_booth_assignments vba
  JOIN booth b ON vba.booth_number = b.booth_number
) AS sub
WHERE rn <= 2;
```

---

39. Rank vendors who sold ‘Apple Pie’ by total revenue and include only those who had inventory uploaded on that date.

```
SELECT *
FROM (
  SELECT cp.vendor_id,
        SUM(cp.quantity * cp.cost_to_customer_per_qty) AS revenue,
        RANK() OVER (ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC) AS
apple_rank
  FROM customer_purchases cp
  JOIN product p ON cp.product_id = p.product_id
  JOIN vendor_inventory vi ON cp.vendor_id = vi.vendor_id AND cp.product_id = vi.product_id
  WHERE p.product_name = 'Apple Pie'
  GROUP BY cp.vendor_id
) AS sub;
```

---

40. For each customer, assign a rank based on total quantity purchased and flag top 3 as VIP using CASE.

```
SELECT *,
    CASE WHEN rnk <= 3 THEN 'VIP' ELSE 'Regular' END AS status
FROM (
  SELECT customer_id,
      SUM(quantity) AS total_quantity,
      RANK() OVER (ORDER BY SUM(quantity) DESC) AS rnk
  FROM customer_purchases
  GROUP BY customer_id
) AS sub;
```

41. (Amazon) Rank products by category and return those consistently ranked in the top 3 in multiple weeks.

```
SELECT *
FROM (
  SELECT p.product_category_id,
      cp.product_id,
      DATE_FORMAT(cp.market_date, '%Y-%m-%d') AS sale_day,
      SUM(cp.quantity) AS total_quantity,
      RANK() OVER (PARTITION BY p.product_category_id, cp.market_date ORDER BY
SUM(cp.quantity) DESC) AS day_rank
  FROM customer_purchases cp
  JOIN product p ON cp.product_id = p.product_id
  GROUP BY p.product_category_id, cp.product_id, cp.market_date
) AS sub
WHERE day_rank <= 3;
```

---

42. (TCS) Use ROW_NUMBER on customer_purchases to detect duplicate same-day purchases by the same customer.

```
SELECT *
FROM (
  SELECT customer_id,
        market_date,
        ROW_NUMBER() OVER (PARTITION BY customer_id, market_date ORDER BY purchase_id) AS rn
  FROM customer_purchases
) AS sub
WHERE rn > 1;
```

---

43. (Infosys) Rank booth assignments per vendor by date and use LAG to find time gaps between assignments.

```
SELECT *,
      DATEDIFF(market_date, prev_date) AS days_between
FROM (
  SELECT vendor_id,
        market_date,
        LAG(market_date) OVER (PARTITION BY vendor_id ORDER BY market_date) AS prev_date
  FROM vendor_booth_assignments
) AS sub;
```

---

44. (Flipkart) Rank vendor revenue per week using RANK, partitioned by week.

```
SELECT *,
      RANK() OVER (PARTITION BY sale_week ORDER BY total_revenue DESC) AS weekly_rank
FROM (
  SELECT vendor_id,
```

```
    WEEK(market_date) AS sale_week,
    SUM(quantity * cost_to_customer_per_qty) AS total_revenue
  FROM customer_purchases
  GROUP BY vendor_id, WEEK(market_date)
) AS sub;
```

---

45. (Walmart) Rank products by number of unique customers and return the top 5 per category.

```
SELECT *
FROM (
  SELECT p.product_category_id,
    cp.product_id,
    COUNT(DISTINCT cp.customer_id) AS unique_customers,
    RANK() OVER (PARTITION BY p.product_category_id ORDER BY COUNT(DISTINCT
cp.customer_id) DESC) AS cust_rank
  FROM customer_purchases cp
  JOIN product p ON cp.product_id = p.product_id
  GROUP BY p.product_category_id, cp.product_id
) AS sub
WHERE cust_rank <= 5;
```

---

46. Rank top 3 customers per vendor by total revenue from purchases.

```
SELECT *
FROM (
  SELECT cp.vendor_id,
    cp.customer_id,
    SUM(cp.quantity * cp.cost_to_customer_per_qty) AS total_revenue,
    RANK() OVER (PARTITION BY cp.vendor_id ORDER BY SUM(cp.quantity *
cp.cost_to_customer_per_qty) DESC) AS revenue_rank
```

```
  FROM customer_purchases cp
  GROUP BY cp.vendor_id, cp.customer_id
) AS sub
WHERE revenue_rank <= 3;
```

---

47. Compute running total of quantity sold per product category using SUM OVER.

```
SELECT p.product_category_id,
    cp.product_id,
    cp.market_date,
    cp.quantity,
    SUM(cp.quantity) OVER (PARTITION BY p.product_category_id ORDER BY cp.market_date) AS running_quantity
FROM customer_purchases cp
JOIN product p ON cp.product_id = p.product_id;
```

---

48. Generate a customer code using CONCAT, LEFT, and rank them within their ZIP code using DENSE_RANK.

```
SELECT *,
    CONCAT(LEFT(customer_first_name, 1), LEFT(customer_last_name, 1), customer_id) AS customer_code,
    DENSE_RANK() OVER (PARTITION BY customer_zip ORDER BY customer_id) AS zip_rank
FROM customer;
```

---

49. Detect products purchased in 3 or more consecutive weeks using RANK and date grouping.

```
SELECT customer_id,
    product_id,
```

```
    WEEK(market_date) AS week_num,
    COUNT(DISTINCT WEEK(market_date)) AS week_count
FROM customer_purchases
GROUP BY customer_id, product_id
HAVING COUNT(DISTINCT WEEK(market_date)) >= 3;
```

---

50. Rank vendors by number of repeat customers (customers who bought more than once).

```
SELECT vendor_id,
    COUNT(DISTINCT customer_id) AS repeat_customers,
    RANK() OVER (ORDER BY COUNT(DISTINCT customer_id) DESC) AS vendor_rank
FROM (
  SELECT vendor_id, customer_id, COUNT(*) AS purchase_count
  FROM customer_purchases
  GROUP BY vendor_id, customer_id
  HAVING COUNT(*) > 1
) AS repeat_data
GROUP BY vendor_id;
```

51. Use ROW_NUMBER to detect if a vendor was reassigned the same booth more than once.

```
SELECT *
FROM (
  SELECT vendor_id,
      booth_number,
      ROW_NUMBER() OVER (PARTITION BY vendor_id, booth_number ORDER BY market_date) AS rn
  FROM vendor_booth_assignments
) AS sub
```

```
WHERE rn > 1;
```

---

52. Rank vendors by the number of distinct booth types they have used.

```
SELECT vba.vendor_id,
    COUNT(DISTINCT b.booth_type) AS booth_type_count,
    RANK() OVER (ORDER BY COUNT(DISTINCT b.booth_type) DESC) AS type_rank
FROM vendor_booth_assignments vba
JOIN booth b ON vba.booth_number = b.booth_number
GROUP BY vba.vendor_id;
```

---

53. Rank customers based on total spend on sunny days using data from market_date_info.

```
SELECT *
FROM (
  SELECT c.customer_id,
      SUM(cp.quantity * cp.cost_to_customer_per_qty) AS total_spend,
      RANK() OVER (ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC) AS
spend_rank
  FROM customer_purchases cp
  JOIN customer c ON cp.customer_id = c.customer_id
  JOIN market_date_info mdi ON cp.market_date = mdi.market_date
  WHERE mdi.market_rain_flag = 0 AND mdi.market_snow_flag = 0
  GROUP BY c.customer_id
) AS sub;
```

---

54. Rank vendor-product pairs by original price using LEFT to format product names.

```
SELECT *,
      LEFT(p.product_name, 10) AS short_name
FROM (
  SELECT vi.vendor_id,
        vi.product_id,
        vi.original_price,
        RANK() OVER (PARTITION BY vi.vendor_id ORDER BY vi.original_price DESC) AS price_rank
  FROM vendor_inventory vi
) AS sub
JOIN product p ON sub.product_id = p.product_id;
```

---

55. Rank products by category where product names start with a vowel.

```
SELECT *
FROM (
  SELECT p.product_id,
        p.product_category_id,
        p.product_name,
        RANK() OVER (PARTITION BY p.product_category_id ORDER BY p.product_name) AS name_rank
  FROM product p
  WHERE LEFT(LOWER(p.product_name), 1) IN ('a', 'e', 'i', 'o', 'u')
) AS sub;
```

---

56. Rank booths per vendor by how frequently they've been assigned.

```
SELECT *
FROM (
  SELECT vendor_id,
        booth_number,
```

```
    COUNT(*) AS times_assigned,
    RANK() OVER (PARTITION BY vendor_id ORDER BY COUNT(*) DESC) AS booth_rank
  FROM vendor_booth_assignments
  GROUP BY vendor_id, booth_number
) AS sub;
```

---

57. Use RANK to get the top 2 customers for each product based on total revenue.

```
SELECT *
FROM (
  SELECT cp.product_id,
      cp.customer_id,
      SUM(cp.quantity * cp.cost_to_customer_per_qty) AS revenue,
      RANK() OVER (PARTITION BY cp.product_id ORDER BY SUM(cp.quantity *
cp.cost_to_customer_per_qty) DESC) AS rnk
  FROM customer_purchases cp
  GROUP BY cp.product_id, cp.customer_id
) AS sub
WHERE rnk <= 2;
```

---

58. Normalize vendor names using TRIM, LEFT, and rank them by booth assignments.

```
SELECT *
FROM (
  SELECT TRIM(LEFT(v.vendor_name, 15)) AS vendor_short,
      COUNT(vba.booth_number) AS booth_count,
      RANK() OVER (ORDER BY COUNT(vba.booth_number) DESC) AS booth_rank
  FROM vendor v
  JOIN vendor_booth_assignments vba ON v.vendor_id = vba.vendor_id
```

```
  GROUP BY TRIM(LEFT(v.vendor_name, 15))
) AS sub;
```

---

59. For Saturdays, rank the first 3 customers per product by quantity purchased.

```
SELECT *
FROM (
  SELECT cp.product_id,
        cp.customer_id,
        cp.market_date,
        SUM(cp.quantity) AS total_qty,
        RANK() OVER (PARTITION BY cp.product_id, cp.market_date ORDER BY SUM(cp.quantity) DESC)
AS rnk
  FROM customer_purchases cp
  JOIN market_date_info mdi ON cp.market_date = mdi.market_date
  WHERE mdi.market_day = 'Saturday'
  GROUP BY cp.product_id, cp.customer_id, cp.market_date
) AS sub
WHERE rnk <= 3;
```

---

60. Identify vendors who had inconsistent booth price levels (i.e., large variance) using RANK and joins.

```
SELECT *
FROM (
  SELECT vba.vendor_id,
        AVG(b.booth_price_level) AS avg_price,
        MAX(b.booth_price_level) - MIN(b.booth_price_level) AS price_variance,
        RANK() OVER (ORDER BY MAX(b.booth_price_level) - MIN(b.booth_price_level) DESC) AS
variance_rank
```

```
  FROM vendor_booth_assignments vba
  JOIN booth b ON vba.booth_number = b.booth_number
  GROUP BY vba.vendor_id
) AS sub
WHERE price_variance > 1;
```